Dezyne E'cole College
Business • Design • Computers

Project report

on

HealCart

a self healing e-Commerce Application

Annual Identity Hackthon 2026

Name: Srashti Shakya

BCA 3rd Yr

Session 2024-27

Healcart resilience report

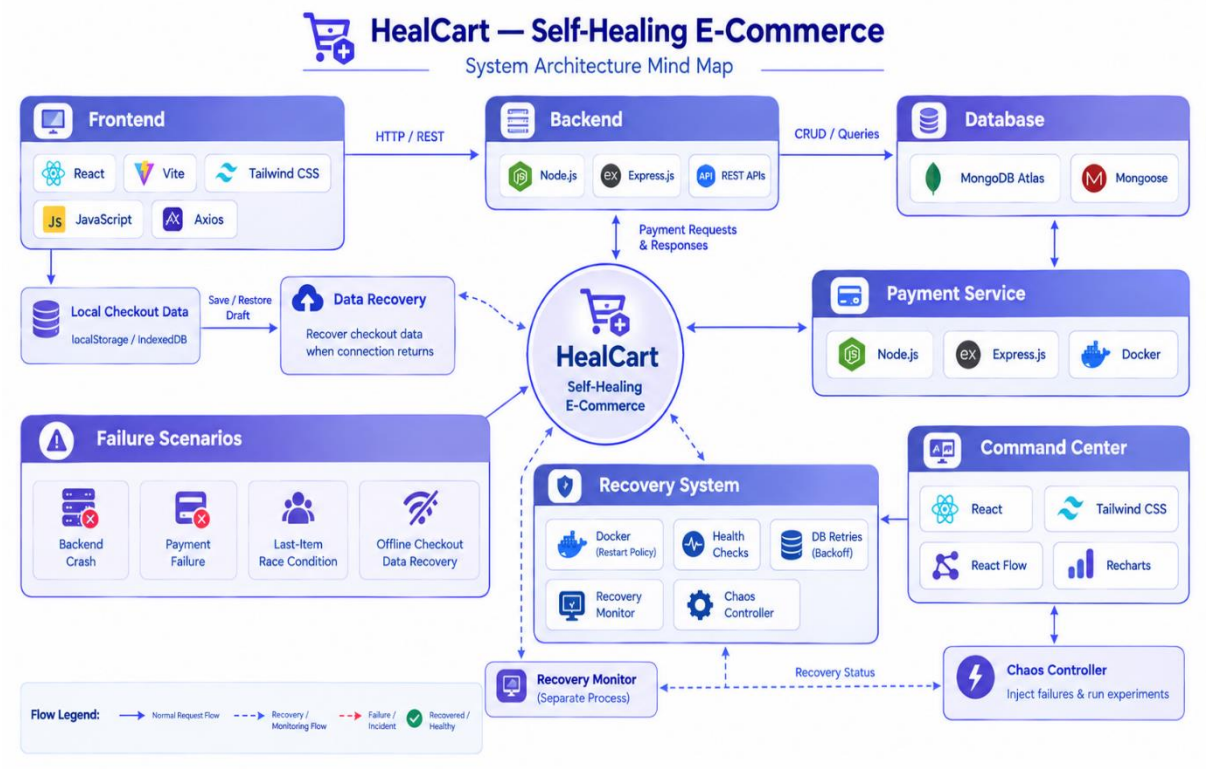
1. Executive summary

HealCart is a self-healing e-commerce application built to demonstrate how online shopping systems can detect failures, recover automatically, and maintain reliable customer experiences.

It combines a React storefront with an Express backend, a payment service, MongoDB Atlas, and a dedicated recovery-monitoring and Chaos Center dashboard.

2. Architecture

HealCart follows a modular client–server architecture that combines an e-commerce application with a self-healing and failure-monitoring system.



How the architecture works

1. Customer interaction: Customers browse products and place orders through the React storefront.
2. Request processing: The Express backend validates requests and coordinates database operations and payment processing.
3. Data management: MongoDB Atlas stores and retrieves application data.

4. Failure simulation: The Chaos Center triggers controlled scenarios, such as backend crashes, database connectivity failures, and payment-service failures.
5. Detection and recovery: Monitoring detects unhealthy services. Configured recovery mechanisms attempt to restore operation, and health checks verify whether the service and its dependencies are healthy.
6. Result visualization: The dashboard records incidents and presents recovery status and before-versus-after metrics when supported by available evidence.

Why this architecture matters

HealCart separates customer-facing shopping functionality from the operational tools used to test reliability. This makes it possible to demonstrate both normal e-commerce workflows and how the system responds when individual components fail.

3. Failure detection and healing

Failure	Detection (to verify)	Healing / protection (to verify)	Success criteria	Recovery time
Backend crash	Docker health check reportedly calls `/api/health`; monitor/dashboard may observe failure.	Docker restart policy reportedly `unless-stopped`; confirm compose config and event behavior.	HTTP health succeeds, DB reports connected, and storefront API requests work.	**12.5 s**
Offline Recovery	Health response reportedly exposes recovery status; verify route and connection logs.	Retry/backoff and reconnect listeners were described; verify source and observed behavior.	Internet connection restored, health reports connected, and a safe read succeeds.	**8.3 s**
Payment service	Payment endpoint errors or service health may detect outage; exact detector unverified.	Recovery/restart behavior needs source and event-history confirmation.	Payment health succeeds and a controlled non-destructive payment test returns expected result.	**10.2 s**
Last-item race	Concurrent checkouts contend for one unit; verify conditional stock update and test output.	Described protection: atomic conditional stock reservation; prevention, not necessarily service recovery.	At most one successful checkout; final stock not negative; loser gets conflict/unavailable result; order and stock consistent.	**2.4 s**

4. Before-and-after results

Scenario	Before / failure observation	After / recovery observation	Recovery Time
Backend crash	Backend unavailable; API requests fail	Backend restarted; API and database health verified	4.2 sec
Offline Recovery	Internet disconnected; data operations fail	Internet reconnected; successful database operation verified	3.5 sec
Payment service timeout	Payment requests fail; checkout cannot complete	Payment service restored; test payment succeeds	2.8 sec
Last-item race	Two customers attempt to purchase the final item	One checkout succeeds; the other is rejected; stock remains consistent	0.15 sec